Data Mining

Data Mining with Databricks

Claudio Sartori

DISI
Department of Computer Science and Engineering – University of Bologna, Italy
claudio.sartori@unibo.it

Learning Objectives

- Understand what Databricks is from a **functional perspective**
- Learn how notebooks work in Databricks
- See how **SQL and Python** interact in the same workflow
- Understand the role of Databricks in a Machine Learning project
- Perform basic hands-on actions in the platform

What is Databricks? (Functional View)

Databricks is a **cloud-based platform** that provides:

- A unified environment for:
  - data analysis
  - machine learning
  - SQL analytics
- Collaborative notebooks
- Integrated experiment tracking (not available with the free edition)

Key idea: Databricks is an *environment*, not just a tool.

What Databricks is NOT

- Not a traditional DBMS
- Not only a Python notebook
- Not only a BI tool
- Not a replacement for transactional databases

Databricks is best seen as a **workspace for data-driven projects**.

Why Databricks for Machine Learning?

Databricks supports the full ML workflow:

1. Data exploration
2. Feature engineering
3. Model training
4. Experiment tracking
5. Result analysis

All steps happen in the **same environment**.

Databricks Notebooks

A Databricks notebook:

- Is a sequence of executable cells
- Supports multiple languages
- Combines:
  - code
  - text (Markdown)
  - visual output

Notebooks are designed for **collaboration and reproducibility**.

Multi-language Notebooks

Databricks notebooks support:

- **Python** (main language for ML)
- **SQL** (data exploration and analytics)
- R, Scala (not used in this course)

Different languages can be used:

- in different cells
- in the same notebook

SQL and Python Together

Typical workflow:

1. Use **SQL** to explore, join, aggregate, filter data
2. Train ML models in Python

This reflects how ML projects work in practice.

From SQL to Machine Learning

- SQL is used to:
  - inspect data
  - compute aggregates
  - check data quality
- Python is used to:
  - build features
  - train models
  - evaluate performance

Databricks makes the transition seamless.

Role of Databricks in an ML Project

Databricks typically sits between:

- data sources (files, databases)
- model consumers (reports, applications)

It acts as:

- an experimentation environment
- a data processing workspace
- an ML development platform

Local Notebook vs Databricks

Local Notebook

- Single-user
- Local environment
- Manual experiment tracking

Databricks

- Collaborative
- Cloud-based
- Integrated tracking

Databricks Free Edition

For this course we use:

- **Databricks Free Edition**
- Free access
- No credit card required

Suitable for:

- learning
- laboratories
- small to medium datasets

Accessing Databricks

Steps:

1. Search for "Databricks Free Edition Sign Up"
2. Create an account (no credit card required – free use with limitations)
3. Access your workspace
4. Open a notebook
5. Load a dataset
6. Run SQL and Python cells

Each student works in a private environment. In real world, the DBMS is usually shared

Loading a Dataset

Typical options:

- use existing DBMS tables
- Upload a CSV file
- Read from provided datasets

The dataset will be used for:

- SQL exploration
- SQL preprocessing
- Machine Learning experiments

Why SQL Before Machine Learning?

SQL is often the fastest way to:

- Verify assumptions about the dataset (schema, ranges, categories)
- Identify data quality issues early (missing values, duplicates, inconsistent codes)
- Build intuition on the target and key predictors
- Define a clean dataset extract for downstream feature engineering

Didactic focus: use SQL as a first-pass lens before any modeling.

Tables and Views: Core Concepts

Table

- Stores data (physically persisted)
- Used as the stable reference for analysis

View

- Stores a query definition (logical layer)
- Useful for reusable filters, joins, and curated subsets

In this lab, we use:

- a table for the raw dataset
- views for curated datasets (optional)

Hands on Databricks

move to Databricks

The Core Question

Where should preprocessing happen?

- In the DBMS / SQL layer?
- In the Python / ML layer?

Key idea:

- There is no single answer
- The decision depends on the type of operation

Case 1: Large Data Volumes

Use SQL when:

- data is large
- data movement is expensive
- data already resides in a DBMS or lakehouse

Reason:

- avoid moving large datasets to Python
- leverage optimized query engines

Case 2: Filtering and Selection

Use SQL for:

- row filtering
- column selection
- removing duplicates

Principle:

- reduce data size as early as possible

Case 3: Aggregations and Joins

Use SQL for:

- joins across tables
- group-by aggregations
- feature construction from relational data

Example:

- number of transactions per customer
- average spending per user

Case 4: Reusable Transformations

Use SQL when transformations must be:

- reused across models
- shared across teams
- consistent over time

Benefit:

- single source of truth
- easier auditing and governance

Case 5: Data Quality and Cleaning

Use SQL for:

- simple missing value handling
- constraint enforcement
- anomaly detection rules

Key idea:

- data should be cleaned close to the source

When Python is Preferable

Use Python when preprocessing is:

- complex or statistical
- tightly coupled with the model
- iterative and experimental

Case 6: ML-Specific Transformations

Use Python for:

- scaling and normalization
- encoding categorical variables
- feature engineering pipelines

Reason:

- these are part of the ML model workflow

Case 7: Integration with ML Pipelines

In scikit-learn:

- preprocessing is part of the pipeline

Example concept:

- imputation + scaling + model

Benefit:

- consistency between training and inference

Case 8: Experimental Work

Use Python when:

- testing ideas quickly
- trying multiple models
- iterating on features

Python is better for:

- flexibility
- rapid experimentation

Best Practice: Hybrid Approach

Modern ML systems use both:

- SQL layer
  - filtering
  - joins
  - aggregations
- Python layer
  - encoding
  - scaling
  - modeling

Typical Pipeline

DBMS / SQL

↓

clean dataset

↓

Python (ML pipeline)

↓

model

Interpretation:

- SQL prepares the data
- Python builds the model

Rule of Thumb

Use SQL if:

- data reduction
- filtering
- aggregation
- simple deterministic logic

Use Python if:

- model-related processing
- statistical transformations
- complex feature engineering

Two-Lines Summary

Move computation to the data unless it is model-centric

In most of the cases something will remain for the model level

Questions?

What is a SQL+Python Workflow in Databricks?

A **hybrid ML pipeline** where:

SQL layer

Data ingestion
Feature engineering
Delta Live Tables (DLT)
Owned by data engineers

Python layer

Model training
Evaluation and tuning
MLflow logging
Owned by data scientists

Key structural issue

Every handoff between layers requires **materializing intermediate results** (Delta tables, temp views, DataFrames) — introducing latency and I/O overhead.

Limitation 1: Context-Switching Overhead

SQL + Python (Databricks)

- Change feature ⇒ re-run SQL pipeline
- Serialize to Delta ⇒ reload in Python
- Errors may originate in either layer
- Iteration cycle is slow

Full Python

- Single execution context
- No serialization boundary
- pandas → sklearn → MLflow in one notebook
- Iteration cycle is fast

Limitation 2: Feature Engineering Expressiveness

SQL is powerful for set-based operations but **awkward** for:

Hard in SQL

- Temporal / sequential features
- Lag features with complex logic
- NLP and text preprocessing
- Graph-based features
- Custom statistical transforms

Natural in Python

- `pandas` rolling windows
- `scipy` / `statsmodels` transforms
- `spaCy`, `transformers` for NLP
- `networkx` for graph features
- Arbitrary procedural logic

Limitation 3: Pipeline Reproducibility & Versioning

A hybrid pipeline is **harder to version atomically**:

- SQL queries may live in separate notebooks or DLT definitions
- Python training code lives in different notebooks or repos
- Reproducing an experiment requires pinning both layers simultaneously
- MLflow does **not** automatically capture SQL transformation history

Full-Python alternative

Tools like **MLflow Projects**, **ZenML**, or **Metaflow** can version the entire pipeline — including feature engineering — as a single reproducible unit.

Limitation 4: Unit Testing

SQL layer — difficult to test

- No native `pytest` equivalent
- Requires Spark context even for small tests
- Mocking tables is cumbersome
- CI/CD integration is non-trivial

Python layer — straightforward

- Standard `pytest` + fixtures
- Lightweight mocks with `unittest.mock`
- Runs locally without cluster
- Easy integration in GitHub Actions / GitLab CI

Consequence

SQL transformations often go **untested** in practice, creating hidden

Limitation 5: Portability

SQL+Python on Databricks

- Tightly coupled to Databricks and Delta Lake
- Cannot run locally for development
- Cluster required even for small experiments
- Migration to another platform is costly

Full Python

- Runs on laptop, any cloud, any CI
- Fast local development loop
- Container-friendly (`Docker`)
- Platform-agnostic

When the Hybrid Workflow Wins

Despite its limitations, SQL+Python on Databricks has **real advantages** in production settings:

Scenario	Why SQL+Python wins
Large-scale data prep on Delta Lake	SQL on Spark is highly optimized
Mixed eng + science teams	Clear layer ownership and separation
Regulatory auditability	SQL is readable by non-programmers
Scheduled production pipelines	DLT + Databricks Workflows orchestration
Incremental / streaming data	Delta Live Tables handles it natively

Summary: Head-to-Head Comparison

Dimension	SQL+Python (Databricks)	Full Python
Iteration speed	Slower	Faster
Expressiveness	Limited in SQL	Full
Testability	Hard	Easy
Portability	Low	High
Reproducibility	Manual effort	Automatable
Scale & production	Strong	Depends on infra
Team collaboration	Good (split)	Requires discipline
Vendor independence	Low	High

Rule of thumb

Full Python for **research and experimentation**; SQL+Python for **production pipelines on large data with mixed teams**.